[AI Native Camp] Day 4 — Context Engineering + Compound Engineering: 만든 걸 복리로 키우는 법

목차

1. 개요
  1.1 서론
  1.2 전체 구조
2. AI 시대의 새로운 업무 운영체제, Claude Code
  2.1 Claude Code의 역할과 중요성
  2.2 Context Engineering: AI가 내 맥락을 이해하게 하는 기술
3. GitHub: AI 시대의 맥락 관리 인프라
  3.1 GitHub의 핵심 기능과 중요성
  3.2 GitHub을 활용한 폴더 구조 설계
4. Compound Engineering: AI와 함께 복리 성장하는 방법
  4.1 Compound Engineering의 개념과 원리
  4.2 Compound Engineering의 풀 사이클과 구체적 이득
  4.3 Session Lab 스킬: Compound Engineering의 실천 도구
5. AI와 함께 일하는 시스템 구축: 여정의 마무리
  5.1 AI Native Camp의 여정 리캡
  5.2 팀에 Compound Engineering 적용하기

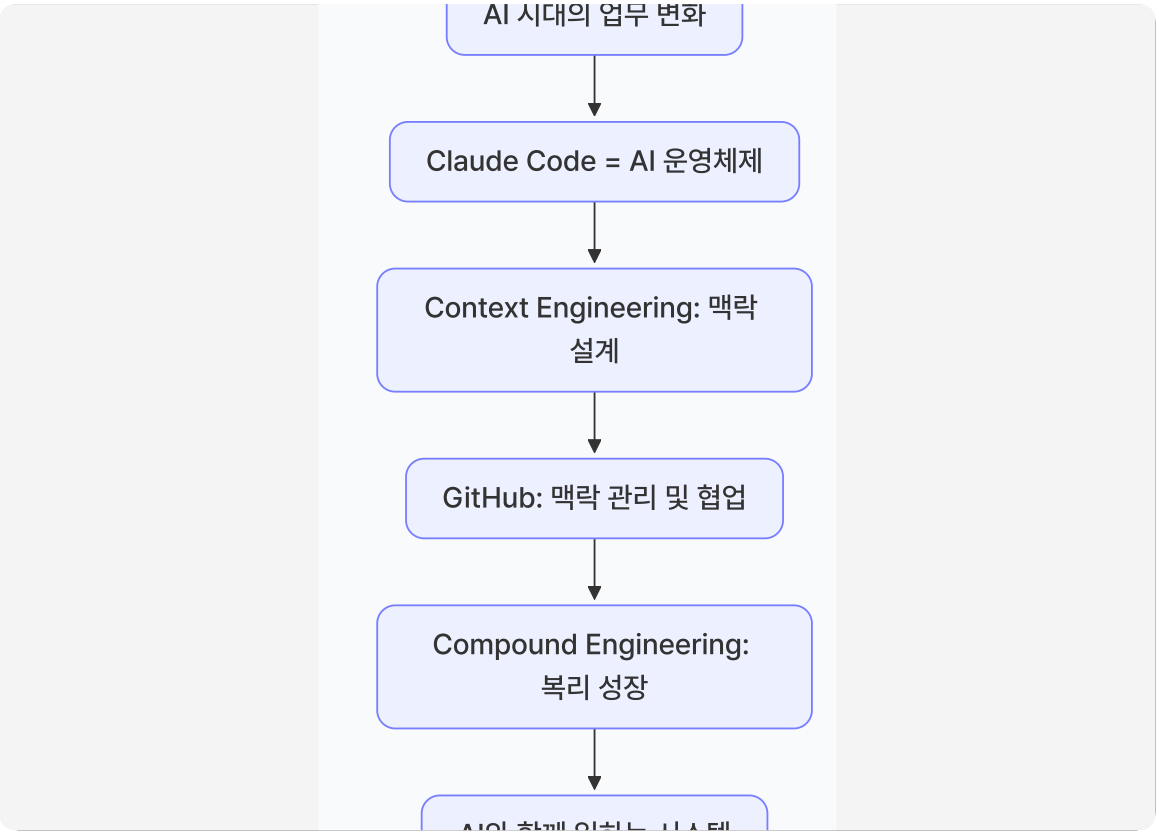
1. 개요

1.1 서론

AI 시대에 접어들면서 우리는 단순히 AI 도구를 사용하는 것을 넘어, AI와 함께 일하는 방식을 근본적으로 변화시켜야 하는 시점에 와 있습니다. 특히, AI를 활용하여 업무 효율을 극대화하고 지속적으로 성장하기 위한 핵심 개념으로 'Context Engineering'과

'Compound Engineering'이 주목받고 있습니다. 이 두 가지 개념은 AI를 단순한 도구가 아닌, 함께 성장하는 파트너로 만드는 데 필수적인 요소입니다.

1.2 전체 구조



2. AI 시대의 새로운 업무 운영체제, Claude Code

2.1 Claude Code의 역할과 중요성

AI 시대의 업무 환경에서 Claude Code는 단순한 도구를 넘어 **'업무 운영체제(OS)'**와 같은 역할을 수행합니다. 기존 컴퓨터의 OS가 하드웨어를 제어하고 앱을 실행하며 파일을 관리하듯이, Claude Code는 AI 환경에서 도구를 제어하고 워크플로우를 실행하며 맥락을 관리하는 핵심적인 역할을 합니다. [A,B]

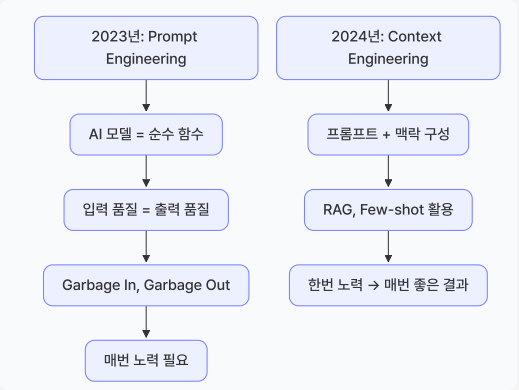
구분	컴퓨터 OS	Claude Code (AI OS)
역할	하드웨어 제어, 앱 실행, 파일 관리	도구 제어, 워크플로우 실행, 맥락 관리

환경	터미널 (내 컴퓨터 위에서 직접 일함)	터미널 (내 컴퓨터 위에서 직접 일함)
기존 AI	ChatGPT/Gemini (대화창 기반)	Claude Code (터미널 기반)
특징	사람이 복사/붙여넣기 필요	AI가 맥락을 읽고 프로세스 직접 실행

- **프롬프트 엔지니어링을 넘어선 환경 세팅**: 과거에는 '프롬프트를 잘 써야 AI가 잘 답한다'는 인식이 강했지만, Claude Code 환경에서는 환경이 잘 세팅되면 대충 물어봐도 AI가 잘 답하는 경험을 할 수 있습니다. [B]
  - **CLAUDE.md**: AI가 참고할 맥락을 설계하여 매번 말하지 않아도 AI가 맥락을 이해하게 합니다. [A,B]
  - **MCP 연결**: 외부 도구(Slack, Notion, 이메일 등)의 데이터를 AI가 직접 읽어 처리할 수 있도록 연결합니다. [A,B]
  - **스킬 자동 실행**: 한마디 명령으로 전체 프로세스를 실행하여 업무 효율을 극대화합니다. [A,B]
- **AI와 함께 일하는 시스템 구축**: Claude Code는 AI가 맥락을 이해하고, 데이터를 읽고, 프로세스를 실행하는 종합적인 시스템을 구축하는 데 핵심적인 역할을 합니다. [A,B]

2.2 Context Engineering: AI가 내 맥락을 이해하게 하는 기술

Context Engineering은 **AI가 사용자의 맥락을 정확하게 이해하고 관리하도록 설계하는 기술**을 의미합니다. 2023년에는 '프롬프트 엔지니어링'이 중요했지만, 2024년부터는 '프롬프트 + 맥락'을 잘 구성하는 것이 중요해졌습니다. [A,B]



- **맥락 전달의 어려움**: 사람 간에도 '지식의 저주' 때문에 맥락 전달이 어렵습니다. 전문가는 비전문가보다 더 나쁜 커뮤니케이터일 수 있으며, 조직 내 지식 공유 실패 비용은 막대합니다. [A,B]
  - **MIT Sloan 연구**: 전문가는 자기 도메인에서 비전문가보다 더 나쁜 커뮤니케이터입니다. [B]
  - **IDC 조사**: 직원 전문 스킬의 42%는 본인만 알고 있습니다. [B]
  - **F500 연간 지식 공유 실패 비용**: $31.5B에 달합니다. [B]
- **Context Engineering의 핵심 교훈**:
  - **프롬프트가 아니라 데이터가 문제**: CLAUDE.md가 부정확하면 AI 결과도 부정확합니다. [A,B]
  - **계획 리뷰가 결과 리뷰보다 효율적**: AI가 만든 2,000줄의 결과물보다 200줄의 계획을 리뷰하는 것이 훨씬 낫습니다. [B]
  - **한번 설계하면 매번 효과**: 맥락을 한번 잘 설계하면, "대충 물어봐도" AI가 잘 답합니다. [B]
- **Context Engineering의 실제 적용**: AI Native Camp에서는 Day 1의 CLAUDE.md 설계와 Day 2의 MCP 연결을 통해 이미 Context Engineering을 실천하고 있었습니다. [B]
  - **CLAUDE.md**: 내 맥락을 설계하는 과정입니다. [B]
  - **MCP 연결**: 외부 도구의 맥락을 AI에 연결하여 맥락을 확장하는 과정입니다. [B]

3. GitHub: AI 시대의 맥락 관리 인프라

3.1 GitHub의 핵심 기능과 중요성

GitHub은 AI 시대에 **AI가 참고할 맥락을 저장하고, 공유하며, 변경 이력을 관리하고, 협업을 리뷰하는 데 필수적인 인프라**입니다. [A,B] 기존의 Google Drive가 단순히 파일을 저장하고 공유하는 역할에 그쳤다면, GitHub은 코드와 AI 관련 자산(CLAUDE.md, 스킬, MCP 설정 등)의 변경 이력을 체계적으로 관리하여 안정성을 보장합니다. [B]

기능	설명	AI 시대의 중요성
Repository (레포지토리)	프로젝트의 '집'으로, CLAUDE.md, 스킬 파일, MCP 설정 등이 저장되는 공간입니다. [A,B]	AI 프로젝트의 모든 자산을 체계적으로 관리하는 기본 단위입니다.
Commit (커밋)	'게임 세이브 포인트'처럼, 잘 작동하는 시점에 변경 사항을 저장하고 왜 변경했는지 메모를 남깁니다. [A,B]	AI가 실수했을 때 이전 버전으로 쉽게 되돌릴 수 있는 안전망 역할을 합니다.
Branch (브랜치)	'원본 안 건드리고 실험하는 복사본'으로, 원본(main)은 항상 안전하게 유지하고 새로운 시도는 브랜치에서 진행합니다. [A,B]	AI 스킬이나 CLAUDE.md를 수정할 때 원본에 영향을 주지 않고 안전하게 실험할 수 있습니다.
Pull Request (PR)	'이렇게 바꿔봤는데 봐줘'라는 의미로, 변경 사항을 원본에 합치기 전에 다른 사람의 검토를 요청하고 피드백을 받아 수정하는 과정입니다. [A,B]	AI 관련 자산의 품질을 높이고 협업을 원활하게 하는 데 필수적입니다.
Pull (풀)	원격 저장소의 최신 변경 사항을 내 로컬 환경으로 가져오는 작업입니다. [A]	여러 사람이 함께 작업할 때, 다른 동료의 업데이트를 놓치지 않고 최신 상태에서 작업을 시작할 수 있도록 합니다.

Push (푸시)	로컬 환경에서 커밋한 변경 사항을 원격 저장소(GitHub)에 반영하는 작업입니다. [A]	내 작업 내용을 팀원들과 공유하고, 원격 저장소에 백업하여 협업을 가능하게 합니다.

- **비개발자를 위한 GitHub**: 과거에는 개발자 전유물로 여겨졌지만, AI 시대에는 비개발자도 GitHub을 활용하여 AI 관련 문서, 스킬, 워크플로우를 관리해야 합니다. [A]
  - AI가 만든 결과물도 한 글자만 바뀌어도 전체 시스템에 영향을 줄 수 있기 때문에, Git과 GitHub을 통한 체계적인 버전 관리가 필수적입니다. [A,B]
  - GitHub은 AI가 천방지축으로 작동할 수 있는 위험을 관리하고, 과거 프로그래밍 수준의 치밀한 관리 시스템을 제공하는 안전망 역할을 합니다. [A,B]

3.2 GitHub을 활용한 폴더 구조 설계

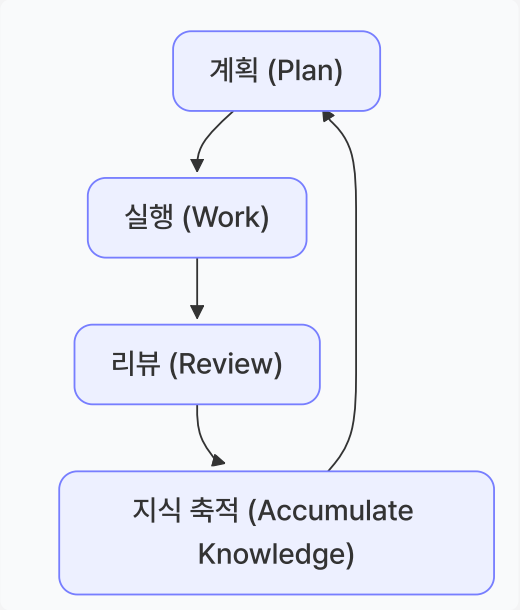
AI와 함께 효율적으로 작업하고 '컴파운딩' 효과를 얻기 위해서는 **GitHub을 기반으로 한 체계적인 폴더 구조 설계가 매우 중요합니다.** 이는 AI가 맥락을 정확히 이해하고, 필요한 정보를 적시에 찾아 활용할 수 있도록 돕는 핵심 요소입니다. [A]

- **모노레포 구조의 장점**: 여러 서비스나 프로젝트를 하나의 레포지토리(모노레포)로 관리하고, 그 안에 각 서비스별 CLAUDE.md를 두는 방식은 AI가 전체 맥락과 개별 서비스 맥락을 동시에 이해하는 데 유리합니다. [A]
- **서브모듈 활용**: 외부에 배포해야 하는 스킬이나 특정 모듈은 서브모듈로 관리하여, 전체 프로젝트의 SOT(Single Source of Truth)와 외부 배포용 자산을 분리하여 관리할 수 있습니다. [A]
- **AI를 위한 폴더 구조**:
  - 각 폴더마다 명확한 목적을 부여하여 AI가 어떤 파일이 어떤 용도로 사용되는지 쉽게 파악하도록 합니다. [A]
  - `CLAUDE.md`는 폴더마다 만들 수 있으며, 해당 폴더의 컨텍스트를 주입하는 역할을 합니다. [A]
  - AI가 특정 파일을 업데이트하거나 읽어야 할 때, 폴더 구조가 맥락에 맞게 설계되어야 혼란을 방지할 수 있습니다. [A]
- **디자인 패턴으로서의 폴더 구조**: 개발 업무에서 디자인 패턴이 폴더 구조에 반영되듯이, 비개발 업무에서도 도메인에 맞는 효율적인 폴더 구조 디자인 패턴을 연구하고 적용하는 것이 중요합니다. [A]

4. Compound Engineering: AI와 함께 복리 성장하는 방법

4.1 Compound Engineering의 개념과 원리

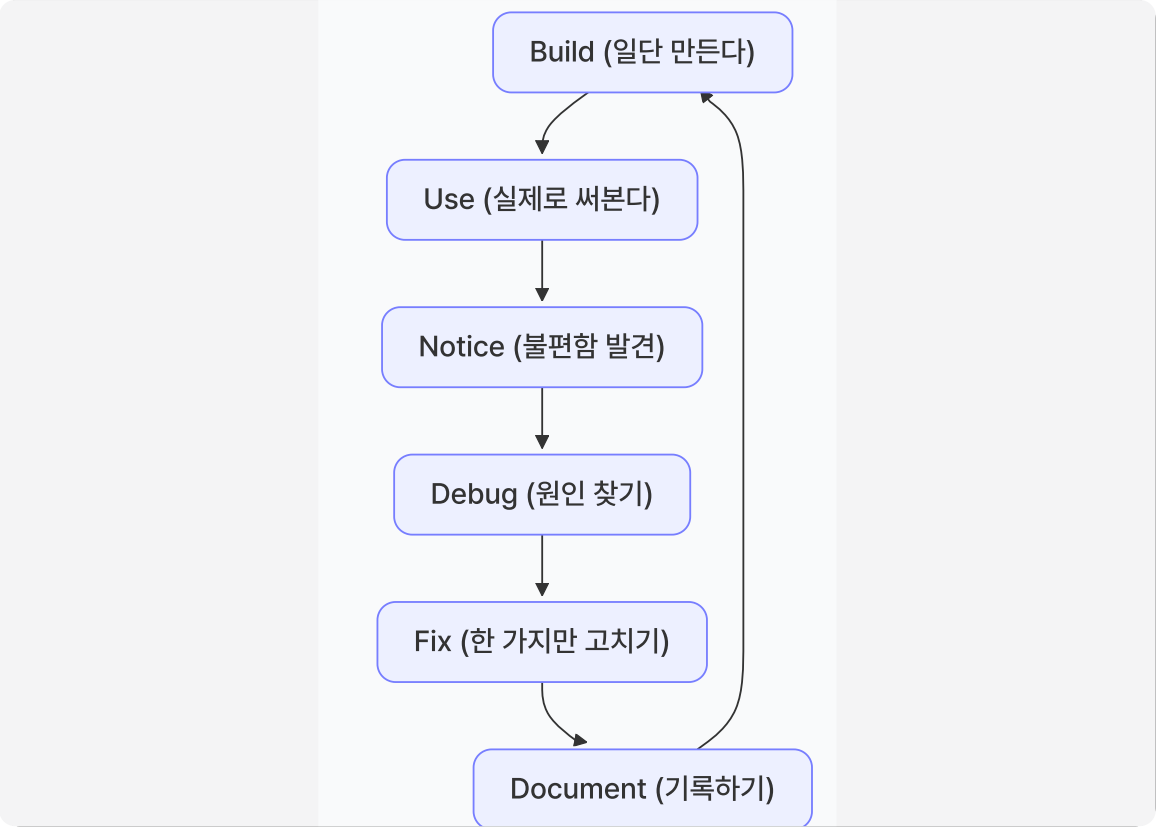
Compound Engineering은 **AI에게 일을 시킬 때마다 AI를 더 똑똑하게 만들어 업무 효율을 복리(Compound Interest)처럼 불리는 방법**입니다. [A,B] 이는 단순히 AI를 한 번 잘 사용하는 것을 넘어, 지속적인 개선과 반복을 통해 AI의 성능과 활용 가치를 극대화하는 철학입니다. [A,B]



- **타이피스트에서 매니저로의 전환**: 과거에는 사람이 직접 문서 작성, 분석, 코딩을 했지만, Compound Engineering을 통해 AI에게 계획을 세워주고, 결과물을 검토하며, 피드백하고 승인하는 매니저 역할로 전환할 수 있습니다. [A,B]
- **설계만 하고 고치지 않으면 낡아감**: Context Engineering으로 AI가 참고할 맥락을 설계했다면, Compound Engineering은 그 맥락을 사용하면서 계속 개선하는 반복의 과정입니다. [A,B]
  - AI가 실수했을 때, 단순히 채팅으로 고쳐달라고 하는 것이 아니라 '왜 실수했는지' 원인을 파악하고 CLAUDE.md에 규칙을 추가하여 다음부터는 같은 실수를 반복하지 않도록 합니다. [A,B]
  - 이는 '실수 방지 노트 만들기'와 같으며, AI를 나만의 완벽한 파트너로 만드는 과정입니다. [B]

4.2 Compound Engineering의 풀 사이클과 구체적 이득

Compound Engineering은 **'일단 만들고 → 쓰면서 → 계속 고치면 → 복리'**라는 철학을 바탕으로, AI와 함께 성장하는 풀 사이클을 통해 구체적인 이득을 제공합니다. [B]

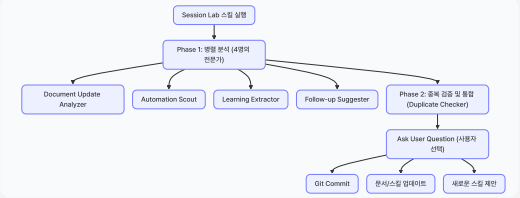


- **풀 사이클의 중요성**: 한 번에 100% 완벽하게 만드는 것이 아니라, 1%씩 100번 개선하는 것이 복리 효과를 가져옵니다. [A,B]
  - **Build**: AI 스킬이나 워크플로우를 일단 만듭니다. [B]
  - **Use**: 실제로 사용해봅니다. [B]
  - **Notice**: 사용 중 불편한 점이나 개선할 부분을 발견합니다. [B]
  - **Debug**: 불편함의 원인을 찾습니다. [B]
  - **Fix**: 발견된 원인 중 한 가지만 집중해서 고칩니다. [B]
  - **Document**: 개선 과정을 기록하여 다음 개선에 활용합니다. [B]
- **구체적인 이득**:
  - **같은 말을 반복하지 않아도 됨**: CLAUDE.md에 한 번 적어두면 AI가 영원히 적용하여 비효율적인 반복을 줄입니다. [B]
  - **혼자서도 팀만큼의 결과**: 약한 모델이라도 좋은 워크플로우(95.1%)를 통해 강한 모델 단독(67.0%)보다 훨씬 높은 정확도를 달성할 수 있습니다. [B]
  - **미래 핵심 역량**: 2025년에는 글로벌 리더의 82%가 AI 에이전트를 팀원으로 도입할 예정이며, Compound Engineering은 이러한 미래 환경에 필수적인 역량입니다. [B]
- **문서화가 성과를 만든다**: Google DORA 2023 연구에 따르면, 질 높은 내부 문서화는 조직 성과에 12.8배의 영향을 미칩니다. [B] AI는 코드보다 문서화 품질을 가장 크게 개선합니다(+7.5%). [B]

- CLAUDE.md에 워크플로우를 잘 정리한 팀일수록 Claude Code가 더 잘 동작합니다. [B]

4.3 Session Lab 스킬: Compound Engineering의 실천 도구

Session Lab 스킬은 **Compound Engineering의 풀 사이클을 자동화하고 촉진하는 핵심 도구**입니다. [A] 이는 AI와의 상호작용을 통해 얻은 경험과 지식을 체계적으로 축적하고 개선하는 루틴을 제공합니다.



- **작동 원리**:
  - **Phase 1 (병렬 분석)**: 세션 내에서 수행된 작업을 4명의 서브 에이전트(전문가)가 동시에 분석합니다. [A]
    - **Document Update Analyzer**: 업데이트가 필요한 문서를 찾아냅니다. [A]
    - **Automation Scout**: 자동화할 수 있는 작업을 제안합니다. [A]
    - **Learning Extractor**: 세션에서 배운 내용을 정리합니다. [A]
    - **Follow-up Suggester**: 다음에 할 일을 제안합니다. [A]
  - **Phase 2 (중복 검증 및 통합)**: Phase 1에서 도출된 내용 중 중복되는 부분을 `Duplicate Checker` 에이전트가 걸러내고 통합합니다. [A]

    - **Ask User Question (사용자 선택)**: 최종적으로 사용자에게 Git 커밋, 문서 업데이트, 새로운 스킬 추가 등 어떤 작업을 수행할지 물어보고 사용자가 직접 선택하여 컴파운딩을 완료합니다. [A]
- **Session Lab의 비유**: 퇴근 전 책상 정리 루틴과 같습니다. [A] 세션을 마칠 때마다 이 스킬을 실행하여 작업 내용을 정리하고, 개선점을 찾아 다음 작업에 반영합니다. [A]
- **Session Lab의 활용 시점**: 세션에서 더 이상 작업할 내용이 없거나, 큰 작업을 완료했다고 생각할 때 Session Lab 스킬을 사용합니다. [A]
    - 이를 통해 문서 추가, 문서 업데이트, 자동화할 작업 발굴, 배운 내용 정리 등 지속적인 지식 축적과 개선이 이루어집니다. [A]
    - 이 과정에서 복리 효과를 체감하며 AI를 더 잘 사용하게 됩니다. [A]

5. AI와 함께 일하는 시스템 구축: 여정의 마무리

5.1 AI Native Camp의 여정 리캡

AI Native Camp는 AI와 함께 일하는 시스템을 구축하기 위한 체계적인 여정을 제공했습니다. [B]

Day	주제	핵심 내용	가져간 것
Day 1	"왜 여기 모였는가"	Claude Code 환경 세팅 및 첫 체험, CLAUDE.md의 중요성 이해 [B]	첫 CLAUDE.md [B]
Day 2	"도구에는 설계 철학이 있다"	6개 도구의 KPI와 설계 철학, MCP 연결을 통한 Claude와 도구 대화 시작 [B]	MCP 연결 환경 [B]
Day 3	"실제로 돌아가는 걸 봤다"	스킬 제작 방법 학습 및 라이브 시연 (슬랙 수집 → Clarify → 과제 자동 생성) [B]	스킬 제작 경험 [B]
Day 4	"만든 걸 내 것으로 만드는 법"	Context Engineering과 Compound Engineering을 통한 지속적인 개선 및 복리 성장 [B]	쓰고, 고치고, 쌓는 과정 [B]

- **커리큘럼 매핑**:
  - **Stage 1 (Day 1-2)**: AI에게 잘 물어보고, AI가 내 맥락을 이해하게 만드는 환경 세팅 단계입니다. [B]
  - **Stage 2 (Day 3)**: AI와 함께 일하는 시스템을 실행하고 체험하는 단계입니다. [B]
  - **Stage 3 (Day 4)**: AI와 함께 일하는 시스템을 유지하고 지속적으로 개선하는 방법을 배우는 단계입니다. [B]

5.2 팀에 Compound Engineering 적용하기

Compound Engineering과 Session Lab 스킬을 팀에 적용할 때는 **지속적인 개선과 지식 축적을 위한 명확한 시스템과 책임 분배가 필요합니다.** [A]

- **CLAUDE.md 업데이트 규칙**:
  - CLAUDE.md는 팀의 핵심 맥락을 담고 있으므로, 업데이트에 대한 책임은 팀을 리드하는 사람이 지는 것이 가장 효율적입니다. [A]
  - 모든 팀원이 CLAUDE.md를 업데이트할 수 있지만, 최종 승인 및 병합은 리더의 책임 하에 이루어져야 합니다. [A]
  - 잦은 업데이트로 인한 충돌(Conflict) 발생 시, 리더가 이를 해결하고 팀의 방향성을 유지하는 역할을 합니다. [A]
- **AI 활용 트래킹 및 대시보드**:
  - 일부 회사에서는 팀원들의 AI 활용 현황을 트래킹하여 효율성을 높입니다. [A]
  - **오픈 텔레메트리(OpenTelemetry)**: 세션 히스토리, 토큰 사용량, 스킬 호출 횟수 등을 수집하여 개인별 AI 활용도를 파악합니다. [A]
  - **대시보드**: 토큰 사용량 및 효율성을 랭크 매겨 대시보드로 공유하여 팀원들이 AI를 더 효율적으로 사용하도록 독려합니다. [A]
  - 소프트한 접근보다는 지능적으로 하드하게 푸시하는 방법이 효과적일 수 있습니다. [A]
- **Compound Engineering의 목표**:
  - 만든 것의 가치는 완성도가 아니라 반복 횟수에 있습니다. (복리) [B]
  - 불편함은 버그가 아니라 개선의 신호입니다. (풀 사이클) [B]
  - 이 사이클을 한 바퀴 돌린 사람은 이미 Compound Engineering을 하고 있는 것입니다. [B]

이처럼 AI와 함께 일하는 시스템을 구축하고 지속적으로 개선하는 과정은 AI 시대의 핵심 역량이며, 여러분의 업무 효율을 복리처럼 성장시키는 중요한 여정이 될 것입니다.

글을 작성하세요. 블록을 추가하려면 '/'를 입력하세요